Introduction to R

What Is R?

R is a programming language and environment designed for statistical computing and data analysis. It was built by statisticians, for statisticians, which means its core syntax and built-in functions reflect the way mathematicians and scientists actually think about data, probability, and computation. Unlike general-purpose languages that treat statistics as an add-on library, R treats it as a first-class concern.

For math majors, R fills a specific role in your workflow. It is the most natural tool for probability and statistics coursework, for verifying hand calculations, for running simulations, and for producing publication-quality statistical graphics. It also functions as a powerful calculator for algebraic and numerical work, handling matrices, systems of equations, and numerical integration in ways that are more concise than equivalent Python code for mathematically structured problems.

This guide covers installation, basic usage as a calculator, core data structures, statistical computing, probability and distributions, matrix operations, plotting, and practical workflows for math coursework.

Table of Contents

1. Installation
2. The RStudio Interface
3. R as a Calculator
4. Variables and Assignment
5. Vectors
6. Matrices
7. Basic Statistics

8. Probability Distributions
9. Simulation and the Law of Large Numbers
10. Data Frames
11. Plotting with Base R
12. Plotting with ggplot2
13. Writing Functions
14. Control Flow
15. Numerical Methods
16. Useful Packages for Math Majors
17. Quick Reference

---

1. Installation

R and RStudio are installed separately. R is the language engine and RStudio is the interface that makes working with R practical. Install R first, then RStudio.

Install R:

Download R from CRAN (the Comprehensive R Archive Network):

> https://cran.r-project.org

Select your operating system and install the most recent release. Accept all defaults during installation.

Install RStudio:

Download RStudio Desktop (free) from Posit:

> https://posit.co/download/rstudio-desktop/

Install RStudio after R is already present on your machine. RStudio detects R automatically.

Verify the installation:

Open RStudio. In the console panel (bottom left), run:

```
R.version.string
```

A string like `"R version 4.4.1 (2024-06-14)"` confirms R is working. You are ready to proceed.

---

2. The RStudio Interface

RStudio divides its workspace into four panels. Understanding what each panel does before writing any code will save significant confusion.

Console (bottom left) This is where R runs commands. Type directly here for quick calculations or testing. Output appears immediately below each command. Anything run here is not saved automatically.

Script editor (top left) This is where you write R scripts, which are plain text files with the `.R` extension. Scripts are saved files that you can run again. Write code here when you want to keep it. Run a line by placing your cursor on it and pressing `Ctrl+Enter` (Windows/Linux) or `Cmd+Enter` (macOS). Run the entire script with `Ctrl+Shift+Enter`.

Environment (top right) This panel shows every variable currently in memory: its name, type, and value. When you assign a variable in the console or a script, it appears here. This panel is useful for tracking what you have defined without printing everything manually.

Files / Plots / Help (bottom right) This panel serves multiple purposes. The Files tab shows your working directory. The Plots tab displays any graphics you produce. The Help tab shows documentation for functions. Type `?function_name` in the console to open documentation for any function directly in this panel.

Setting your working directory:

R reads and writes files relative to a working directory. Set it to the folder where your scripts and data live:

```
setwd("/path/to/your/folder")
```

Or use the RStudio menu: **Session > Set Working Directory > Choose Directory**.

---

3. R as a Calculator

The most immediate use of R is as a calculator. Open the console and start computing.

Arithmetic:

```
2 + 3         # 5
10 - 4        # 6
3 * 7         # 21
22 / 7        # 3.142857
2^10          # 1024
17 %% 5       # 2  (modulo)
17 %/% 5      # 3  (integer division)
```

Mathematical functions:

```r
sqrt(144)         # 12
abs(-7.5)         # 7.5
exp(1)            # 2.718282 (e)
log(exp(1))       # 1  (natural log by default)
log(100, 10)      # 2  (log base 10)
log2(32)          # 5  (log base 2)
factorial(6)      # 720
choose(10, 3)     # 120 (binomial coefficient)
```

Trigonometry (arguments in radians):

```r
sin(pi / 2)       # 1
cos(pi)           # -1
tan(pi / 4)       # 1
asin(1)           # pi/2
acos(-1)          # pi
atan(1)           # pi/4
```

Constants:

```r
pi                # 3.141593
exp(1)            # e = 2.718282
Inf               # infinity
-Inf              # negative infinity
NaN               # not a number (e.g. 0/0)
NA                # missing value
```

Rounding:

```r
round(3.14159, 2)     # 3.14
floor(3.9)            # 3
ceiling(3.1)          # 4
trunc(3.9)            # 3 (toward zero)
```

R respects standard order of operations. Use parentheses liberally to make expressions unambiguous:

```
2 + 3 * 4          # 14
(2 + 3) * 4        # 20
```

---

4. Variables and Assignment

R uses `←` as the primary assignment operator. You will also see `=` used for assignment in some contexts, but `←` is the conventional style in R code.

```
x ← 5
y ← 3.14
name ← "real analysis"
flag ← TRUE
```

Print a variable by typing its name:

```
x          # 5
```

Or use `print()` explicitly:

```
print(x)   # 5
```

Variable names are case-sensitive. `x` and `X` are different variables. Names can contain letters, numbers, periods, and underscores, but must start with a letter or a period.

```
epsilon ← 0.001
delta.bound ← 0.005
```

```r
step_size <- 0.01
```

Remove a variable from the environment:

```r
rm(x)
```

Remove everything from the environment:

```r
rm(list = ls())
```

---

5. Vectors

Vectors are the fundamental data structure in R. A vector is an ordered sequence of values all of the same type. Almost everything in R is built on vectors, including scalars (which are just vectors of length 1).

Creating vectors:

```r
v <- c(1, 2, 3, 4, 5)
w <- c(10, 20, 30)
```

`c()` stands for combine or concatenate.

Sequences:

```r
1:10                      # 1 2 3 4 5 6 7 8 9 10
seq(0, 1, by = 0.25)     # 0.00 0.25 0.50 0.75 1.00
seq(0, 1, length.out = 5) # same result
rep(0, times = 5)         # 0 0 0 0 0
rep(c(1, 2), times = 3)  # 1 2 1 2 1 2
```

Vector arithmetic:

Arithmetic operations on vectors are applied element-wise. This is one of R's most useful features.

```
x <- c(1, 2, 3, 4, 5)
x + 10          # 11 12 13 14 15
x * 2           # 2 4 6 8 10
x^2             # 1 4 9 16 25
sqrt(x)         # 1.000 1.414 1.732 2.000 2.236
```

Operations between two vectors of the same length are also element-wise:

```
a <- c(1, 2, 3)
b <- c(4, 5, 6)
a + b           # 5 7 9
a * b           # 4 10 18
```

Indexing:

R uses 1-based indexing. The first element is at index 1.

```
x <- c(10, 20, 30, 40, 50)
x[1]            # 10
x[3]            # 30
x[c(1, 3, 5)]  # 10 30 50
x[2:4]          # 20 30 40
x[-1]           # 20 30 40 50  (everything except index 1)
```

Logical indexing:

```
x <- c(3, 7, 1, 9, 4)
x[x > 4]        # 7 9  (elements greater than 4)
```

```r
x[x %% 2 == 0] # 4    (even elements)
```

Useful vector functions:

```r
x <- c(4, 1, 7, 2, 9, 3)

length(x)       # 6
sum(x)          # 26
prod(x)         # 1512
mean(x)         # 4.333
median(x)       # 3.5
var(x)          # variance
sd(x)           # standard deviation
min(x)          # 1
max(x)          # 9
range(x)        # 1 9
sort(x)         # 1 2 3 4 7 9
rev(x)          # 3 9 2 7 1 4
cumsum(x)       # cumulative sum
cumprod(x)      # cumulative product
which(x > 5)    # indices where condition is TRUE
```

---

6. Matrices

R has strong native support for matrix operations, which makes it natural for linear algebra coursework.

Creating a matrix:

```r
A <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, ncol = 3)
```

By default, R fills matrices column by column. To fill row by row:

```r
A <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2, byrow = TRUE)
#      [,1] [,2] [,3]
# [1,]    1    2    3
# [2,]    4    5    6
```

Dimensions:

```r
nrow(A)      # 2
ncol(A)      # 3
dim(A)       # 2 3
```

Indexing:

```r
A[1, 2]      # row 1, column 2
A[1, ]       # entire first row
A[, 2]       # entire second column
```

Matrix arithmetic:

```r
B <- matrix(c(1, 0, 0, 1), nrow = 2)   # 2x2 identity

A <- matrix(c(1, 2, 3, 4), nrow = 2)
B <- matrix(c(5, 6, 7, 8), nrow = 2)

A + B        # element-wise addition
A * B        # element-wise multiplication (NOT matrix
multiplication)
A %*% B      # matrix multiplication
```

Use `%*%` for matrix multiplication. The `*` operator multiplies element-wise, which is a common source of error.

Transpose:

```
t(A)
```

Determinant:

```
det(A)
```

Matrix inverse:

```
solve(A)
```

Solving a linear system Ax = b:

```
A ← matrix(c(2, 1, 1, 3), nrow = 2)
b ← c(5, 10)
x ← solve(A, b)
x            # solution vector
```

Eigenvalues and eigenvectors:

```
result ← eigen(A)
result$values     # eigenvalues
result$vectors    # eigenvectors (as columns)
```

Other matrix functions:

```
diag(3)                     # 3x3 identity matrix
diag(c(1, 2, 3))            # diagonal matrix with 1, 2, 3 on
diagonal
diag(A)                     # extract diagonal of A
crossprod(A)                # t(A) %*% A, computed efficiently
norm(A, type = "F")         # Frobenius norm
```

Rank and null space via MASS:

```
library(MASS)
Null(A)        # null space of A
fractions(solve(A))  # display inverse as fractions
```

---

7. Basic Statistics

R's core statistical functions operate directly on vectors.

Summary statistics:

```
x ← c(12, 15, 14, 10, 18, 22, 11, 16, 13, 19)

mean(x)        # arithmetic mean
median(x)      # median
var(x)         # sample variance
sd(x)          # sample standard deviation
min(x)         # minimum
max(x)         # maximum
range(x)       # min and max
quantile(x)    # quartiles
quantile(x, 0.9)  # 90th percentile
IQR(x)         # interquartile range
```

The summary function:

```
summary(x)
```

Prints the five-number summary (min, Q1, median, Q3, max) plus the mean in one call. Run this first whenever you encounter a new dataset.

Covariance and correlation:

```
x ← c(1, 2, 3, 4, 5)
y ← c(2, 4, 5, 4, 6)

cov(x, y)      # covariance
cor(x, y)      # Pearson correlation coefficient
```

For a matrix of pairwise correlations across multiple variables:

```
data_matrix ← cbind(x, y)
cor(data_matrix)
```

Frequency tables:

```
grades ← c("A", "B", "A", "C", "B", "A", "B", "A")
table(grades)
prop.table(table(grades))   # proportions instead of counts
```

---

8. Probability Distributions

R has built-in functions for every major probability distribution. For each distribution, there are four functions following a consistent naming pattern:

Prefix	Purpose	Example
`d`	Probability density or mass function	`dnorm`
`p`	Cumulative distribution function	`pnorm`
`q`	Quantile function (inverse CDF)	`qnorm`
`r`	Random samples	`rnorm`

Normal distribution:

```
dnorm(0)                # density at x = 0 for standard normal
pnorm(1.96)             # P(X ≤ 1.96) for standard normal, approx
0.975
qnorm(0.975)            # z-score with 97.5% below it, approx 1.96
rnorm(10)               # 10 random draws from standard normal
rnorm(10, mean = 5, sd = 2)   # non-standard normal
```

Uniform distribution:

```
dunif(0.5, min = 0, max = 1)  # density at 0.5
punif(0.5, min = 0, max = 1)  # P(X ≤ 0.5) = 0.5
runif(10, min = 0, max = 1)   # 10 uniform random draws
```

Binomial distribution:

```
dbinom(3, size = 10, prob = 0.5)   # P(X = 3) for Bin(10, 0.5)
pbinom(3, size = 10, prob = 0.5)   # P(X ≤ 3)
rbinom(10, size = 10, prob = 0.5)  # 10 random draws
```

Poisson distribution:

```
dpois(4, lambda = 3)    # P(X = 4) for Poisson(3)
ppois(4, lambda = 3)    # P(X ≤ 4)
rpois(10, lambda = 3)   # 10 random draws
```

Exponential distribution:

```
dexp(1, rate = 2)       # density at x = 1
pexp(1, rate = 2)       # P(X ≤ 1)
rexp(10, rate = 2)      # 10 random draws
```

t, chi-squared, and F distributions:

```
pt(-2, df = 9)          # P(T ≤ -2) for t with 9 degrees of freedom
qt(0.025, df = 9)       # 2.5th percentile of t(9)
pchisq(5, df = 3)       # P(X ≤ 5) for chi-squared with 3 df
qf(0.95, df1 = 2, df2 = 10)   # 95th percentile of F(2, 10)
```

Computing probabilities manually:

```
# P(1 < X < 2) for standard normal
pnorm(2) - pnorm(1)

# P(X > 1.645) for standard normal
1 - pnorm(1.645)
# or equivalently:
pnorm(1.645, lower.tail = FALSE)

# P(|X| > 1.96) for standard normal (two-tailed)
2 * pnorm(-1.96)
```

Setting a random seed for reproducibility:

When using random number generation, set a seed first so your results can be reproduced exactly:

```
set.seed(42)
rnorm(5)    # always produces the same five numbers with seed 42
```

---

9. Simulation and the Law of Large Numbers

Simulation is one of the most powerful tools R offers for building intuition about probabilistic results. Instead of working through a proof analytically, you can

simulate the phenomenon, observe it numerically, and then return to the proof with a clearer picture of what it is saying.

Example: verifying the law of large numbers

The law of large numbers states that the sample mean of i.i.d. random variables converges to the true mean as the sample size grows. Simulate this for a standard normal distribution with mean 0:

```
set.seed(1)
n_values <- c(10, 100, 1000, 10000, 100000)

for (n in n_values) {
  sample_mean <- mean(rnorm(n))
  cat("n =", n, "  sample mean =", round(sample_mean, 5), "\n")
}
```

As `n` increases, the sample mean gets closer to 0.

Example: estimating pi by Monte Carlo

A classic simulation: generate random points in the unit square and count how many fall inside the unit circle. The fraction approximates pi / 4.

```
set.seed(2)
n <- 1000000
x <- runif(n, -1, 1)
y <- runif(n, -1, 1)
inside <- x^2 + y^2 <= 1
pi_estimate <- 4 * mean(inside)
pi_estimate    # should be close to 3.14159
```

Example: simulating a random walk

```r
set.seed(3)
n <- 1000
steps <- sample(c(-1, 1), size = n, replace = TRUE)
position <- cumsum(steps)
plot(position, type = "l", main = "Random Walk", xlab = "Step", ylab
= "Position")
```

Example: central limit theorem

The CLT says that the sample mean of i.i.d. draws is approximately normal for large n, regardless of the underlying distribution. Simulate with an exponential distribution (which is skewed):

```r
set.seed(4)
n_sims <- 10000
n <- 50
sample_means <- replicate(n_sims, mean(rexp(n, rate = 1)))

hist(sample_means, breaks = 50, probability = TRUE,
     main = "Distribution of Sample Means (Exponential, n=50)",
     xlab = "Sample Mean")
curve(dnorm(x, mean = 1, sd = 1/sqrt(n)), add = TRUE, col = "red",
lwd = 2)
```

The histogram of sample means follows the normal curve closely, even though the underlying distribution is exponential.

---

10. Data Frames

A data frame is R's structure for tabular data. Each column is a vector (and all values in a column are the same type), but different columns can have different types. A data frame is what you work with when your data has multiple variables.

Creating a data frame:

```r
students <- data.frame(
  name = c("Alice", "Bob", "Carol"),
  score = c(88, 74, 95),
  passed = c(TRUE, TRUE, TRUE)
)
```

Inspecting a data frame:

```r
head(students)          # first 6 rows
tail(students)          # last 6 rows
nrow(students)          # number of rows
ncol(students)          # number of columns
dim(students)           # rows and columns
str(students)           # structure: types and sample values
summary(students)       # summary statistics for each column
```

Accessing columns:

```r
students$score          # the score column as a vector
students[, "score"]     # same result
students[, 2]           # same result by column index
```

Accessing rows:

```r
students[1, ]           # first row
students[students$score > 80, ]   # rows where score > 80
```

Adding a column:

```r
students$grade <- c("B", "C", "A")
```

---

11. Plotting with Base R

R has a built-in plotting system that produces graphs with minimal code. It is useful for quick visualizations during problem-solving.

Basic plot:

```
x <- seq(-3, 3, length.out = 300)
y <- dnorm(x)
plot(x, y, type = "l", main = "Standard Normal Density",
     xlab = "x", ylab = "Density", col = "steelblue", lwd = 2)
```

Plot types:

`type` argument	Result
`"l"`	Line
`"p"`	Points
`"b"`	Both points and lines
`"h"`	Histogram-like vertical lines
`"s"`	Step function

Adding to an existing plot:

```
curve(dt(x, df = 5), add = TRUE, col = "red", lwd = 2)
legend("topright", legend = c("Normal", "t(5)"), col =
c("steelblue", "red"), lwd = 2)
abline(v = 0, lty = 2)   # vertical dashed line at x = 0
abline(h = 0)             # horizontal line at y = 0
```

Histogram:

```r
x <- rnorm(1000)
hist(x, breaks = 30, probability = TRUE,
     main = "Histogram of Normal Samples", xlab = "x")
curve(dnorm(x), add = TRUE, col = "red", lwd = 2)
```

Boxplot:

```r
group_a <- rnorm(50, mean = 0)
group_b <- rnorm(50, mean = 1)
boxplot(group_a, group_b, names = c("Group A", "Group B"),
        main = "Comparison", ylab = "Value")
```

Scatter plot:

```r
x <- runif(100)
y <- 2 * x + rnorm(100, sd = 0.3)
plot(x, y, main = "Scatter Plot", xlab = "x", ylab = "y", pch = 16,
col = "gray40")
abline(lm(y ~ x), col = "red", lwd = 2)   # regression line
```

Saving a plot to a file:

```r
pdf("my_plot.pdf", width = 7, height = 5)
plot(x, y)
dev.off()
```

Replace `pdf()` with `png()` or `svg()` for other formats.

---

12. Plotting with ggplot2

`ggplot2` is a plotting package that produces more polished graphics than base R and follows a consistent grammar. It is the standard choice for any plot you intend to present or publish.

Install and load:

```
install.packages("ggplot2")   # only needed once
library(ggplot2)
```

The basic structure:

Every `ggplot2` plot is built in layers. Start with `ggplot()`, specify the data and aesthetic mappings, then add geometry layers.

```
ggplot(data, aes(x = variable1, y = variable2)) +
  geom_point()
```

Scatter plot:

```
df <- data.frame(x = runif(100), y = runif(100))

ggplot(df, aes(x = x, y = y)) +
  geom_point(color = "steelblue", size = 2) +
  labs(title = "Scatter Plot", x = "x", y = "y") +
  theme_minimal()
```

Line plot of a function:

```
x_vals <- seq(-4, 4, length.out = 300)
df <- data.frame(x = x_vals, y = dnorm(x_vals))

ggplot(df, aes(x = x, y = y)) +
  geom_line(color = "steelblue", linewidth = 1) +
```

```
  labs(title = "Standard Normal Density", x = "x", y = "Density") +
  theme_minimal()
```

Histogram:

```
df <- data.frame(x = rnorm(1000))

ggplot(df, aes(x = x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
                 fill = "steelblue", color = "white", alpha = 0.7) +
  stat_function(fun = dnorm, color = "red", linewidth = 1) +
  labs(title = "Histogram with Normal Overlay", x = "x", y =
"Density") +
  theme_minimal()
```

Plotting multiple functions:

```
x_vals <- seq(-4, 4, length.out = 300)
df <- data.frame(
  x = rep(x_vals, 3),
  y = c(dnorm(x_vals), dt(x_vals, df = 3), dt(x_vals, df = 10)),
  distribution = rep(c("Normal", "t(3)", "t(10)"), each = 300)
)

ggplot(df, aes(x = x, y = y, color = distribution)) +
  geom_line(linewidth = 1) +
  labs(title = "Normal vs. t Distributions", x = "x", y = "Density")
+
  theme_minimal()
```

---

13. Writing Functions

Writing your own functions in R lets you avoid repeating code and makes your scripts readable and testable.

Basic function syntax:

```
function_name <- function(argument1, argument2) {
  result <- argument1 + argument2
  return(result)
}
```

Example: a function for the normal PDF

```
normal_pdf <- function(x, mu = 0, sigma = 1) {
  coefficient <- 1 / (sigma * sqrt(2 * pi))
  exponent <- -((x - mu)^2) / (2 * sigma^2)
  return(coefficient * exp(exponent))
}

normal_pdf(0)             # evaluate at x = 0, default mu = 0, sigma
= 1
normal_pdf(0, mu = 2, sigma = 0.5)
```

Arguments with `=` have default values and are optional when calling the function.

Example: computing a Riemann sum

```
riemann_sum <- function(f, a, b, n = 1000) {
  x <- seq(a, b, length.out = n + 1)
  dx <- (b - a) / n
  midpoints <- (x[-1] + x[-(n+1)]) / 2
  return(sum(f(midpoints)) * dx)
}

riemann_sum(sin, 0, pi)          # should be close to 2
riemann_sum(function(x) x^2, 0, 1)  # should be close to 1/3
```

Vectorized functions:

R functions that use standard arithmetic and built-in functions are already vectorized. They accept vectors as input and return vectors as output with no extra work:

```
square <- function(x) x^2
square(c(1, 2, 3, 4, 5))    # 1 4 9 16 25
```

---

14. Control Flow

if / else:

```
x <- 7

if (x > 5) {
  print("greater than 5")
} else if (x == 5) {
  print("equal to 5")
} else {
  print("less than 5")
}
```

for loop:

```
total <- 0
for (i in 1:10) {
  total <- total + i
}
total    # 55
```

while loop:

```r
n <- 1
while (n < 100) {
  n <- n * 2
}
n    # 128
```

Avoiding loops with vectorization:

In R, explicit loops are often slower and more verbose than vectorized alternatives. Prefer vector operations and the `apply` family of functions when working with arrays and data frames.

```r
# loop version
squares <- numeric(10)
for (i in 1:10) squares[i] <- i^2

# vectorized version (preferred)
squares <- (1:10)^2
```

sapply: apply a function to each element of a vector

```r
sapply(1:5, function(x) x^2)    # 1 4 9 16 25
```

apply: apply a function across rows or columns of a matrix

```r
A <- matrix(1:9, nrow = 3)
apply(A, 1, sum)    # row sums
apply(A, 2, sum)    # column sums
```

---

15. Numerical Methods

R includes built-in tools for numerical integration, root finding, and optimization that are directly useful in courses on numerical analysis, differential equations, and applied mathematics.

Numerical integration:

```
result <- integrate(function(x) sin(x), lower = 0, upper = pi)
result$value      # 2.0 (exact answer is 2)
result$abs.error  # estimated absolute error
```

`integrate()` uses adaptive quadrature and works on any function R can evaluate.

```
# Integrate the standard normal density from -Inf to 1.96
integrate(dnorm, lower = -Inf, upper = 1.96)$value   # about 0.975
```

Root finding:

Find the zero of a function on a given interval using `uniroot()`:

```
# Find sqrt(2) as the root of x^2 - 2 = 0 on [1, 2]
result <- uniroot(function(x) x^2 - 2, interval = c(1, 2))
result$root       # 1.414214
```

Optimization:

Find the minimum of a function using `optimize()` (one variable) or `optim()` (multiple variables):

```
# Find the minimum of (x - 3)^2 on [0, 10]
result <- optimize(function(x) (x - 3)^2, interval = c(0, 10))
```

```
result$minimum    # 3
result$objective  # 0 (the minimum value)
```

Solving differential equations with deSolve:

```
install.packages("deSolve")
library(deSolve)

# dy/dt = -y, y(0) = 1 (solution: y = e^(-t))
ode_system <- function(t, y, params) list(-y)
times <- seq(0, 5, by = 0.1)
initial <- c(y = 1)
solution <- ode(y = initial, times = times, func = ode_system, parms
= NULL)
plot(solution, main = "dy/dt = -y", ylab = "y(t)")
```

---

16. Useful Packages for Math Majors

Install packages with `install.packages("package_name")`. Load them in each session with `library(package_name)`.

Package	Purpose
`ggplot2`	Publication-quality graphics
`MASS`	Linear algebra utilities, statistical methods
`deSolve`	Solving ordinary differential equations
`pracma`	Numerical analysis: integration, root finding, interpolation
`Matrix`	Sparse matrices and advanced linear algebra
`combinat`	Combinatorics functions
`stats`	Built-in, no install needed: regression, hypothesis testing, distributions
`polynom`	Polynomial arithmetic and calculus
`numbers`	Number theory: primes, GCD, LCM, modular arithmetic

Installing multiple packages at once:

```
install.packages(c("ggplot2", "MASS", "deSolve", "pracma"))
```

Checking if a package is already installed:

```
"ggplot2" %in% installed.packages()[, "Package"]
```

---

17. Quick Reference

Arithmetic and math functions:

Task	Command
Square root	`sqrt(x)`
Absolute value	`abs(x)`
Natural log	`log(x)`
Log base 10	`log10(x)`
Exponential	`exp(x)`
Factorial	`factorial(n)`
Binomial coefficient	`choose(n, k)`
Modulo	`x %% y`
Integer division	`x %/% y`

Vectors:

Task	Command
Create vector	`c(1, 2, 3)`

Task	Command
Sequence	`seq(a, b, by = s)`
Repeat	`rep(x, times = n)`
Length	`length(x)`
Sum	`sum(x)`
Cumulative sum	`cumsum(x)`
Sort	`sort(x)`
Index by condition	`x[x > 0]`

Matrices:

Task	Command
Create matrix	`matrix(data, nrow, ncol)`
Transpose	`t(A)`
Matrix multiply	`A %*% B`
Determinant	`det(A)`
Inverse	`solve(A)`
Solve Ax = b	`solve(A, b)`
Eigenvalues	`eigen(A)$values`
Identity matrix	`diag(n)`

Statistics:

Task	Command
Mean	`mean(x)`
Median	`median(x)`
Variance	`var(x)`
Standard deviation	`sd(x)`
Quantiles	`quantile(x)`

Task	Command
Correlation	`cor(x, y)`
Five-number summary	`summary(x)`

Distributions:

Distribution	Density	CDF	Quantile	Random
Normal	`dnorm`	`pnorm`	`qnorm`	`rnorm`
Uniform	`dunif`	`punif`	`qunif`	`runif`
Binomial	`dbinom`	`pbinom`	`qbinom`	`rbinom`
Poisson	`dpois`	`ppois`	`qpois`	`rpois`
Exponential	`dexp`	`pexp`	`qexp`	`rexp`
t	`dt`	`pt`	`qt`	`rt`
Chi-squared	`dchisq`	`pchisq`	`qchisq`	`rchisq`
F	`df`	`pf`	`qf`	`rf`

Numerical methods:

Task	Command
Numerical integration	`integrate(f, lower, upper)`
Root finding	`uniroot(f, interval)`
Optimization	`optimize(f, interval)`
Set random seed	`set.seed(n)`